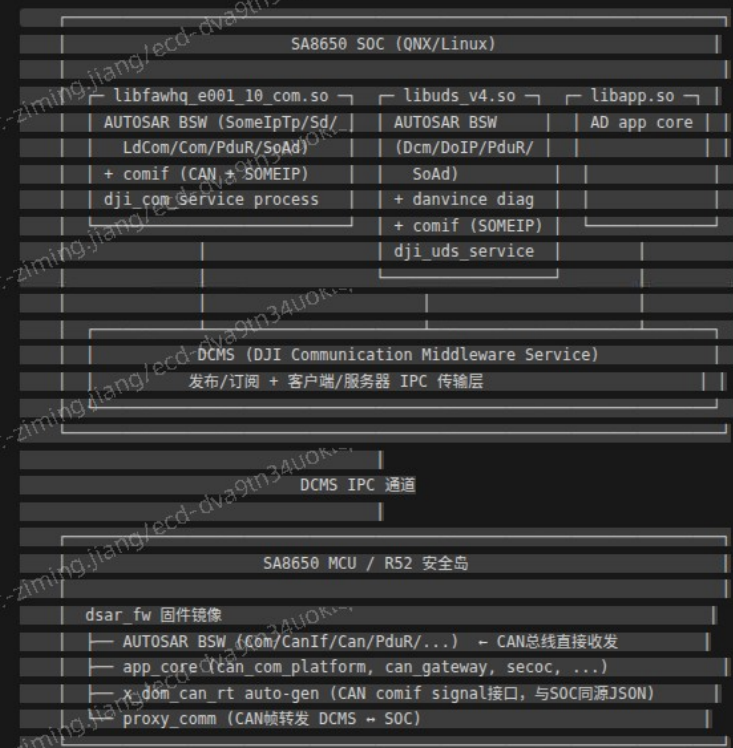


- 下面是从零开始、覆盖SOC侧（COM+UDS）和FW/MCU侧的完整总结，按信号流向追踪。

## 一、总体架构概览



## 二、SOC COM 域 — libfawhq\_e001\_10\_com.so

### 2.1 编译组成

链接行 consys\_bf.cmake:213:

```
libfawhq_e001_10_com.so = --whole-archive (  
  DSAR_APP_LIB_CDD          - plat_bf_cdd.so (动态库, 包含 x_dom_someip_rt SDK)  
  dcms_dcms                 - DCMS IPC 传输层  
  APP_CORE_LIB              - app_core 业务  
  oem_feature                - OEM 功能层  
  x_dom_can_rt_gen_lib_com   - [CAN comif static lib]  
  x_dom_someip_rt_gen_lib_com - [SOME/IP comif static lib]  
  autosar_com               - AUTOSAR BSW: SomeIpTp/Sd/LdCom/Com/PduR/SoAd  
  oem_proxy                  - OEM proxy  
) --no-whole-archive
```

### x\_dom\_can\_rt\_gen\_lib\_com 源码构成

x\_dom\_can\_rt\_gen\_com/CMakeLists.txt:

```
aux_source_directory(${X_DOM_CAN_RT_SRC} ...) - SDK: msg_route_soc.cpp, sig_route_soc.cpp, x_dom_can_rt_soc.cpp, p2p_parser.cpp  
aux_source_directory(当前目录) - 自动生成: x_dom_can_rt_cfg.cpp, x_dom_can_rt_sigif.cpp, x_dom_can_rt_dbg1.cpp
```

编译时宏定义来自 x\_dom\_can\_rt\_gen\_com/.../x\_dom\_can\_rt\_cfg.h:

- XDOMCANRT\_DJI\_COM\_SERVICE\_EN = 1 — COM 角色
- XDOMCANRT\_READ\_MCU\_RX\_SIGS\_EN = 1 — 接收 MCU RX 信号
- XDOMCANRT\_READ\_MCU\_TX\_SIGS\_EN = 1 — 接收 MCU TX 信号
- XDOMCANRT\_WRITE\_SIGS\_TO\_MCU\_EN = 1 — 向 MCU 写信号
- XDOMCANRT\_MSG\_ROUTE\_EN = 1 — 消息路由引擎使能

自动生成的核心数据结构:

- msg\_route\_mcu\_tx\_rx\_msgs\_map — {(channel, canid) - CanMsg\*}, 4条TX+33条RX=37条消息。每个 CanMsg 缓存着原始CAN帧数据
- msg\_route\_write\_to\_mcu\_msgs\_map — 写入 MCU 的消息缓存
- msg\_route\_write\_to\_mcu\_msg\_id\_pairs\_map — 按周期分组的写入消息ID列表
- SigHdl<phy\_t, factor, offset> — 信号编码/解码处理器 (Intel/Motorola 格式)。例如:

```
// VDC DecelerationReq AEB 信号: start_bit=64, len=10, factor=0.01, offset=-10.21  
SigHdl<float, uint16_t, float, float> VDC_DecelerationReq_AEB_VDC_1 default_mcu_tx_sighdl(  
  (float)0.01, (float)0.01, (float)10.21, 64, 10, false, mcu_tx_msg_VDC_1 default_mcu_tx_msg_VDC_1
```

```
(float)0.01, (float)0.01, (float)-10.21, ..., 64, 10, false, MCU_tx_msg_VDC_1 default 0x118, ...);
```

- `sig_route_read_tx_rx_sigs_dji_com_service_t` — 转发给 UDS/AD 的打包信号结构体
- `sig_route_write_to_mcu_sigs_dji_com_service_t` — 从 UDS/AD 收到的写入信号结构体

## x\_dom\_someip\_rt\_gen\_lib\_com 源码构成

`x_dom_someip_rt_gen_com`:

- `x_dom_someip_rt_cfg.cpp` — 路由表: 125 个服务 → `SvcItemInfo` (接收和发送映射)
- `svc_route_recv_svc_info_uds_app / svc_route_recv_svc_info_ad_app` — `NodeRecvInfo`列表
- 调用 `x_dom_someip_rt_set("com_service_en", true)` — COM 是服务端

## autosar\_com 源码构成

`microsar_config_com.cmake`:

- BSW 模块: `SomeIpTp`, `Sd`, `LdCom`, `Com`, `PduR`, `SoAd`, `IpduM`, `Nm`, `UdpNm`, `E2E`, `Crc`
- 关键行 40-41: `if(CONFIG_X_DOM_SOMEIP_RT_COM) add_definitions(-DX_DOM_SOMEIP_RT_COM_EN)`
- 移除 SIP 原生 `Cdd_SomeIpTp.c`, 替换为项目特定 `Cdd_SomeIpTp.c`
- `Cdd_SomeIpTp.c` 引用了 `x_dom_someip_rt_gen_com` 目录下的 `x_dom_someip_rt_cfg.h` 头文件

## 2.2 COM 域的完整调用链路

### 链路 A: CAN 信号流入 (MCU → SOC COM)

【HW】CAN 总线

→ MCU CAN 控制器接收 CAN 帧  
→ MCU 内部 AUTOSAR CanIf → PduR → Com (进程内)  
→ MCU 通过 DCMS 转发给 SOC:  
主题 "/x\_dom\_can\_rt/mcu\_tx\_rx\_msgs"

——【IPC 边界】——

→ SOC COM: `msg_route_mcu_tx_rx_msgs_topic_cbk()` [`msg_route_soc.cpp:490`]  
→ 遍历接收到的 `can_msg_base_with_status_t` 数组  
→ `mcu_can_msg_queue.push(can_msg)` [`msg_route_soc.cpp:498`]  
→ `msg_route_mcu_tx_rx_can_msgs_proc()` [`msg_route_soc.cpp:93`]  
→ `while (queue_size > 0):`  
→ `mcu_can_msg_queue.pop(can_msg)`  
→ `msg_route_can_msg_cb_ctx->set_recv_msg()` [`msg_route_soc.cpp:132`]  
→ `CanMsgCbItem::call()` — 用户注册的回调  
→ 在 `msg_route_mcu_tx_rx_msgs_map` 中查找 `CanMsg*`  
→ `can_msg_ptr->set_data(data, dlc, status)` [`msg_route_soc.cpp:165`]  
→ 缓存原始 CAN 帧数据到 `CanMsg::data_buf[]`  
→ `can_msg_ptr->add_upd_cnt()` — 统计更新次数  
→ `sig_route_send_mcu_tx_rx_sigs()` [自动生成] [`msg_route_soc.cpp:508`]  
→ 从所有 `SigHdl` 关联的 `CanMsg` 缓存中读取信号  
→ 通过 `factor/offset` 将原始值解码为物理值  
→ 填充 `sig_route_read_tx_rx_sigs_dji_com_service_t` 结构体  
→ `dcms_mcu_topic_send("/x_dom_can_rt/read_sigs_app")` → UDS/AD 进程

### 链路 B: CAN 信号流出 (SOC COM → MCU)

SOC COM: 每 50ms 周期任务 `x_dom_can_rt_main_func()` [`x_dom_can_rt_soc.cpp:36`]  
→ `msg_route_main_function()` [`msg_route_soc.cpp:452`]  
→ `msg_route_write_msgs_to_mcu_proc()` [`msg_route_soc.cpp:241`]  
→ 按周期遍历 `msg_route_write_to_mcu_msg_id_pairs_map`  
→ 从 `msg_route_write_to_mcu_msgs_map` 读取 `CanMsg` 缓存数据  
→ 打包为 `can_msg_t` 二进制数组  
→ `dcms_mcu_topic_send("/x_dom_can_rt/write_to_mcu_msgs")` [`msg_route_soc.cpp:311`]  
——【IPC 边界】——  
→ MCU: 接收 DCMS 主题 → AUTOSAR CanIf → CAN 控制器 → 总线发送

### 链路 C: SOME/IP 服务流入 (外部 ECU → COM → UDS/AD 跨域)

【外部 ECU SOME/IP 网络包】

→ `SoAd` → `SomeIpTp`  
→ `Cdd_SomeIpTp.c` 回调 (DaVinci 生成)

【分支1 — danvince 路径 (无 comif 宏保护)】

→ `SOMEIP_DEBUG_INFO(a..)` // 仅调试打印  
→ `LdCom → PduR → Com` → 同进程内 SWC 消费 (数据不出 COM 域)

【分支2 — comif 路径 (#ifdef X\_DOM\_SOMEIP\_RT\_COM\_EN)】

→ `x_dom_someip_rt_rx_handle(server_id, data, len)` [`x_dom_someip_rt.cpp:87`]  
→ `svc_route_rx_handle(id, data, len)` [`service_route.cpp:80`]  
→ 在 `svc_route_recv_svc_info_map_ptr` 中根据 `id` 查找 `SvcItemInfo`  
→ `item_ptr->set_origin_data(id, data, len)` [`service_route.cpp:94`]  
→ 缓存原始 SOME/IP 数据  
→ 调用 `sig_route_read_tx_rx_sigs_dji_com_service_t`



```
→ 如果是即时服务:
→ svc_route_immediately_pub() [service_route.cpp:545]
→ 遍历 svc_route_recv_svc_info_list_ptr(每个目标节点)
→ check_and_send_svc_via_dcms(id, data, topic name) [x_dom_someip_rt_com.h]
→ SvcDataPackHdl: append_item(id, data, len, recv_count)
→ dcms_mcu_topic_send(topic, packed_blob)
```

每50ms周期发布 (非即时服务):

```
x_dom_someip_rt_main_func() *com_app_comif" [x_dom_someip_rt.cpp:43]
→ svc_route_main_function() (com_service_enabled=true) [service_route.cpp:507]
→ svc_route_cycle_pub() [service_route.cpp:561]
→ 对每个 NodeRecvInfo (uds_app, ad_app):
→ check_and_send_svc_list_via_dcms_on_timeout(
    *svc_route_recv_svc_info_map, svc_list, topic_name)
→ 检查每个 SvcItemInfo 的超时时间
→ SvcDataPackHdl 批量打包 → dcms_mcu_topic_send()
主题 "/x_dom_someip_rt/com_to_app/uds_app" (给 UDS)
主题 "/x_dom_someip_rt/com_to_app/ad_app" (给 AD)
```

链路 D: SOME/IP 服务流出 (UDS/AD → COM → 外部 ECU)

```
SOC COM: svc_route_app_to_com_topic_cbk() [service_route.cpp:107]
→ DCMS 主题 "/x_dom_someip_rt/app_to_com" 来自 UDS/AD 的请求
→ SvcDataUnPackHdl::get_next_data(id, data, len, recv_count)
→ item_ptr->fwd_svc_via_someip() [x_dom_someip_rt_com.h]
→ 调用 AUTOSAR SomeIpTp 发送接口 → SOME/IP → 外部 ECU
```

## 三、SOC UDS 域 — libuds\_v4.so

### 3.1 编译组成

链接行 [consys\\_bf.cmake:237](#):

```
libuds_v4.so = --whole-archive (
    DSAR_APP_LIB_CDD          - plat_bf_cdd.so (动态库, 含 x_dom_someip_rt SDK)
    dcms_dcms                  - DCMS IPC 传输层
    UDSOINIP_TARGET_v4         - UDS over IP 协议栈
    diagproxy_ge               - 诊断代理 (通过 comif 收发 OTA 服务)
    x_dom_someip_rt_gen_lib_uds_v4 - [SOME/IP comif static lib]
    autosar_uds_v4             - AUTOSAR BSW: Dcm/DoIP/PduR/SoAd
) --no-whole-archive
PRIVATE: DCOS_LIBRARIES, mini_dcos
```

注意: fawhq\_e001\_10 当前 `CONFIG_X_DOM_CAN_RT_UDS_V4=OFF`, 所以 UDS v4 不链接 `x_dom_can_rt_gen_lib_uds_v4`。

`x_dom_someip_rt_gen_lib_uds_v4` 源码构成

`x_dom_someip_rt_gen_uds/CMakeLists.txt`:

```
aux_source_directory(当前目录...) →
x_dom_someip_rt_cfg.cpp          - 路由表 + 服务注册
x_dom_someip_rt_sigif_get.cpp    - 服务数据 Get + 回调机制
x_dom_someip_rt_sigif_set.cpp    - 服务数据 Set + 即时发送
x_dom_someip_rt_dbg.cpp          - 调试接口
```

4 个 OTA 服务的具体文件:

`x_dom_someip_rt_cfg.cpp` 定义:

- 3 个接收 SvcItemInfo (svc\_route\_recv\_svc\_info\_map):
  - `PS_FM_OTASelfDL_Mgmt_HAD_JO_OTASelfDL_RRSelfDownloadCmd` — 独立下载命令
  - `PS_FM_OTASelfDL_Mgmt_HAD_JO_OTASelfDL_RRSelfDownloadStatus` — 下载状态
  - `PS_FM_OTASelfDL_Mgmt_HAD_JO_OTASelfDL_RRSelfDownloadScript` — 下载脚本
- 4 个发送 SvcItemInfo (svc\_route\_send\_svc\_info\_map):
  - 以上 3 个的 TX 方向 + `OTASelfDL_NotifyOnChgSelfDLPacketScript` — 下载脚本包通知
- `svc_route_write_someip_init(cbk)` — 注册 DCMS 回调接收 `"/x_dom_someip_rt/com_to_app/uds_app"` 主题
- `x_dom_someip_rt_prod_cfg()` — 配置 `app_service_en=true`, `topic_node_name="uds_app_comif"`

`x_dom_someip_rt_sigif_get.cpp` — 接收信号获取:

```
// 每服务一对: Get函数 + SetCallback函数
int32_t SigIf...Get(cpp_PS_SelfDLCmd_stptr *pData) {
    ret_ptr = someip_recv_svc_info...RRSelfDownloadCmd_HAD_JO
    get_usr_data_ext<cpp_PS_SelfDLCmd_stptr>(
        cpp_PS_SelfDLCmd_stptr_DataOffset, pData,
        cpp_PS_SelfDLCmd_stptr_Deserialize); // 反序列化
    // 用 com_recv_count 判断是否新数据
    if (count != item.get_com_recv_count())
```

```
return XDOMSOMEIPRT_SVC_RT_NO_ERROR; // 有新数据
else
return XDOMSOMEIPRT_SVC_RT_TIMEOUT; // 数据未更新
}
```

SigIf\_OnRecvData(id) — COM 转发数据到达时触发，在 SvcItemInfo 的 set\_origin\_data() 中回调：

```
void SigIf_OnRecvData(uint32_t id) {
switch (id) {
case X_DOM_SOMEIP_RT_SERVER_ID...RX:
if (cb != nullptr) {
SigIf...Get(&data); // 反序列化
cb(&data, ctx); // 回调应用层
}
break;
}
}
```

x\_dom\_someip\_rt\_sigif\_set.cpp — 发送信号写入：

```
int32_t SigIf...Set(const cpp_PS_OTAResponse_default_enum *pData) {
someip_send_svc_info...Set.set_usr_data_ext<>(..., pData, Serialize); // 序列化
if (item.is_immediate_svc()) {
item.fwd_svc_via_dcms("/x_dom_someip_rt/app_to_com", false); // 即时通过 DCMS 发送
}
}
```

## autosar\_uds\_v4 源码构成

microsar\_config\_uds.cmake:

- BSW 模块: Dcm, Dem, DoIP, PduR, SoAd, BswM, ComM, Fee, Fls, NvM, MemIf, Crc, Os
- 不含 LdCom, Sd, SomeIpTp (属于 COM 域)
- OEM 功能源文件: security\_cert\_manager, DiagCalib, uds\_proxy\_ota, FactoryParam, SecurityAccess, DcmApp\_Reset, UpdateAdapter, SubscribeProcess
- 手动加入的适配文件:
  - DiagnosticService.c — 诊断服务实现
  - djl\_rtc.c — RTE 胶水
  - Dcm\_Callout\_Stubs.c — Dcm 回调存根
  - DiagSwc\_Callback.c — 诊断 SWC 回调 (DID 读写)
  - Appl\_DoIP.c — DoIP 应用层
  - Dcm\_Mcu\_Proxy.c — MCU 代理

## diagproxy\_ge 源码构成

DiagProxy\_config.cmake:

diagproxy\_ge = DiagProxy\_Com.cpp

包含头文件: x\_dom\_someip\_rt\_gen\_uds/, x\_dom\_can\_rt\_gen\_uds/

- 为 C 代码 (UDS AUTOSAR 栈是 C) 提供 C++ comif 接口的桥梁
- 例如 DiagProxy\_GetOTARRSelDownloadCmdSig() 调用 SigIf...Get() 获取 OTA 下载命令

## 3.2 UDS 域的四条信号流入链路

### 链路 1: DoIP 诊断请求 (danyince 路径 — 非 comif)

```
【外部诊断仪】DoIP 以太网请求
→ SoAd (Socket Adapter) — TCP/UDP socket 收发
DoIP (Diagnostic over IP) — 路由激活、诊断消息解封装
→ PduR (PDU Router) — PduR_DoIP → PduR_Dcm 路由
→ Dcm (Diagnostic Communication Manager) — 解析 UDS SID:
SID 0x22 ReadDataByIdentifier
SID 0x2E WriteDataByIdentifier
SID 0x31 RoutineControl
SID 0x34/0x36/0x37 传输/下载
SID 0x10/0x11/0x27/0x3E 会话/安全/测试
→ Dcm_Callout_Stubs.c:
Dcm_GetSessChg() — 会话切换回调
Dcm_GetSecurityLevelChg() — 安全等级变更
Dcm_ActiveSession() — 默认会话激活
Dcm_DiagnosticService_0x38() — 文件传输服务
→ DiagSwc_Callback.c:
DiagDid_0xF187_ReadData() — 读取 ECU 硬件号
DiagDid_0xF190_ReadData() — 读取 VIN/VIN 码
DiagDid_0xF190_WriteData() — 写入 VIN
DiagDid_0xF198_ReadData() — 读取维修店代码
...(数十个DID 读写函数通过 rte_config.DID_Read_ScvgRop 回调机制)
→ DiagnosticService.c:
所有 UDS 服务的共同处理: 文件 CRC、复位、安全证书管理等
djl_rtc.c: omf
```



RTE 接口注册: DID 读写、调度、IO 控制

## 链路 2: CAN 信号从 MCU 流入 UDS (comif CAN 路径)

```
【MCU收集CAN帧】
→ MCU: DCMS 发送 "/x_dom_can_rt/mcu_tx_rx_msgs"
----- [IPC]
→ SOC COM: msg_route_mcu_tx_rx_msgs_topic_cbk()
CanMsg 缓存, SigHdl 读取, 打包为 sig_route_read_tx_rx_sigs_dji_com_service_t
→ dcms_mcu_topic_send("/x_dom_can_rt/read_sigs_uds")
----- [IPC]
→ SOC UDS: sig_route_mcu_rx_sigs_topic_cbk() [sig_route_soc.cpp:68]
→ sig_route_update_mcu_tx_rx_sigs_from_com() [x_dom_can_rt_cfg_uds.cpp:53, 自动生成]
→ 验证 hash_value 和 size
memcpy 到本地全局结构体 sig_route_read_tx_rx_sigs_dji_uds_service
→ 设置 update_flag = true
```

但目前 fawhq\_e001\_10 CONFIG\_X\_DOM\_CAN\_RT\_UDS\_V4=0FF, 所以 UDS 进程中的 CAN 数据接收注册代码不执行。UDS 当前主要通过 SOME/IP comif 获取车辆数据。

## 链路 3: SOME/IP 服务从 COM 流入 UDS (comif SOME/IP 路径)

```
【COM 进程】
→ Cdd_SomeIpTp.c 中 #ifdef X_DOM_SOMEIP_RT_COM_EN 的125个服务
→ x_dom_someip_rt_rx_handle() → svc_route_rx_handle()
→ set_origin_data() 缓存
→ 每50ms 或即时: svc_route_cycle_pub() / svc_route_immediately_pub()
→ SvcDataPackHdl::append_item() - 批量打包多个服务
dcms_mcu_topic_send("/x_dom_someip_rt/com_to_app/uds_app")
----- [IPC]
→ SOC UDS: svc_route_com_to_app_topic_cbk() [service_route.cpp:147]
→ SvcDataUnPackHdl::get_next_data(id, data, len, com_recv_count)
→ 在 svc_route_recv_svc_info_map_ptr 中查找 SvcItemInfo
→ item_ptr->set_com_recv_count(com_recv_count)
→ item_ptr->set_origin_data(id, data, len) - 更新缓存
→ 内部调用 m_recv_func(id) → SigIf_OnRecvData(id) - 自动生成的回调触发
→ switch(service_id):
    → SigIf_PS_FM_OTASelfDL ... Get(&data) - 反序列化
    → user_callback(&data, ctx) - 通知应用层
```

应用层也可以通过轮询方式直接调用 SigIf ... Get() 而不注册回调:

```
// 轮询模式 - 业务代码主动调用
PS_SelfDLCmd strt_data;
int32_t ret = SigIf_PS_FM_OTASelfDL_Mgmt_HAD_JO_OTASelfDL_RRSelfDownloadCmd_HAD_JO_Get(&data);
if (ret == XDOMSOMEIPRT_SVC_RT_NO_ERROR) {
    // 新数据到达, 处理 data
} // else XDOMSOMEIPRT_SVC_RT_TIMEOUT - 无新数据
```

## 链路 4: UDS 发送数据回 COM (comif 反向路径)

```
【业务层调用 SigIf_Set】
DiagProxy_SetOTARRSelfDownloadCmdSig(Ack_Data) [DiagProxy_Com.cpp:37]
→ SigIf_PS_FM_OTASelfDL ... Set(&convertVal) [sigif_set.cpp:44]
→ someip_send_svc_info ... Set.set_usr_data_ext<>(..., pData, Serialize)
    → 序列化
    → 如果是即时服务:
        item.fwd_svc_via_dcms("/x_dom_someip_rt/app_to_com", false)
        → SvcDataPackHdl 打包
        → dcms_mcu_topic_send("/x_dom_someip_rt/app_to_com")
----- [IPC]
→ SOC COM: svc_route_app_to_com_topic_cbk() [service_route.cpp:107]
→ SvcDataUnPackHdl::get_next_data()
→ item_ptr->fwd_svc_via_someip() - 通过 AUTOSAR SomeIpTp 发送给外部 ECU
```

# 四、FW/MCU 侧

## 4.1 编译组成

FW/MCU 使用 Makefile 构建, 所有 .c/.cpp 编译为目标文件后直接链接为固件 ELF。

fw 侧 x\_dom\_can\_rt 自动生成文件 (与 SOC 侧同源 JSON x\_dom\_can\_rt\_fawhq\_e001\_10.json 生成):

```
dsar_fw/product/faw/fawhq_e001_10/x_dom_can_rt_config/
├── x_dom_can_rt_cfg.h - 功能开关宏
├── x_dom_can_rt_cfg.c - 消息过滤器表、写入信号表、VIP 消息区
├── x_dom_can_rt_sigif.h - SigIf 函数声明
```

- └─ x\_dom\_can\_rt\_sigif.c ─ 信号 Set 实现 (通过 AUTOSAR Com\_SendSignal)
- └─ x\_dom\_can\_rt\_dbg.h ─ 调试函数声明
- └─ x\_dom\_can\_rt\_dbg.c ─ 调试 shell 命令

与 SOC 侧编译的关键区别

|         | SOC COM 侧                          | FW/MCU 侧                               |
|---------|------------------------------------|----------------------------------------|
| SDK 头文件 | x_dom_can_rt_soc.h                 | x_dom_can_rt_mcu.h                     |
| 信号处理    | sig_handle_soc.hpp (C++ 模板)        | sig_handle_mcu.h (C)                   |
| 消息路由    | msg_route_soc.h (DCMS + CanMsg 缓存) | msg_route_mcu.h (AUTOSAR Com)          |
| 信号输出    | dcms_mcu_topic_send() IPC          | Com_SendSignal(signal_id, data) CAN 总线 |
| 数据缓存    | CanMsg 类 (msg_route_soc.h)         | MCU 消息区 buf 数组                         |
| 编译配置    | XDOMCANRT_DJI_COM_SERVICE_EN=1     | SigErr_SendSignal = Com_SendSignal     |

FW 侧 x\_dom\_can\_rt\_cfg.h 定义的功能开关:

```
#define XDOMCANRT_READ_MCU_RX_SIGS_EN 1
#define XDOMCANRT_READ_MCU_TX_SIGS_EN 1
#define XDOMCANRT_WRITE_SIGS_TO_MCU_EN 1
#define XDOMCANRT_READ_MCU_TX_MSG_EN 1
#define XDOMCANRT_READ_MCU_RX_MSG_EN 1
#define XDOMCANRT_WRITE_SIGS_FROM_MCU_TO_VIP_EN 1
#define SigErr_SendSignal(signal_id, signal_data) Com_SendSignal(signal_id, signal_data)
```

x\_dom\_can\_rt\_cfg.c 定义的数据结构:

- msg\_route\_rx\_msg\_filters[36] ─ 36 个 CAN RX 消息过滤器 (channel, canid, cycle, timeout, e2e)
- msg\_route\_tx\_msg\_filters[4] ─ 4 个 CAN TX 消息过滤器
- msg\_route\_write\_sig\_tbl[37] ─ 37 个写入信号表 (canid, com\_sig\_id, start\_bit, len, is\_big\_endian, e2e)
- mcu\_write\_to\_vip\_msg\_\*\_buf[] ─ VIP 消息缓冲区

4.2 FW 侧的调用链路

FW 链路 A: CAN 信号采集并转发给 SOC

```
【CAN 总线】
└─ MCU CAN 控制器 (CAN Driver) 接收 CAN 帧
└─ AUTOSAR CanIf → PduR → Com
└─ Com_RxIndication() 通知信号更新
└─ MCU x_dom_can_rt: 从 AUTOSAR Com 读取信号
└─ SigIf_Get_xxx 函数 (通过 sig_handle_mcu.h 实现) 从 Com 缓存读取
└─ 组装 DCMS 消息 → 主题 "/x_dom_can_rt/mcu_tx_rx_msgs"

└─ ─────────────────── 【IPC】 ───────────────────
└─ SOC COM: msg_route_mcu_tx_rx_msgs_topic_cbk()
└─ ... (见上文链路 A)
```

FW 链路 B: 接收 SOC 的 CAN 写入请求

```
SOC COM: dcms_mcu_topic_send("/x_dom_can_rt/write_to_mcu_msgs")
└─ ─────────────────── 【IPC】 ───────────────────
└─ MCU: 接收 DCMS 消息
└─ 解析 can_msg_t 数组, 提取 (channel, canid, data)
└─ 调用 SigIf_Set_xxx(signal_phy) ─ x_dom_can_rt_sigif.c 中自动生成
└─ sig_if_set_raw_value(start_bit, len, ..., &msg_buf, &sig_raw_data)
└─ 将物理值转为原始值, 填入消息缓冲区
└─ Com_SendSignal(com_sig_id, signal_data) ─ 通过 AUTOSAR Com 发送
└─ AUTOSAR Com → CanIf → CAN Driver → CAN 总线
```

例如, AD 功能通过 SOC COM 请求控制 EPS (x\_dom\_can\_rt\_sigif.c:69):

```
uint8_t SigIf_Set_ADV_AutoParkingModeReq_oADV_2_0x100_odefault(uint8_t *signal_phy) {
    retval = sig_if_set_raw_value(
        9, 3, // start bit=9, len=3
        0, 0, CAN_VALUE_TYPE_UNSIGNED, // Intel 格式
        mcu_write_to_vip_msg_ADV_2_default_0x0x100_buf, // 目标消息缓冲区
        sizeof(...), &sig_raw_data);
    return (uint8_t)retval;
}

// 消息缓冲区内容随后通过 Com_SendSignal 发送到 CAN 总线
```

FW 链路 C: SOC → VIP (MCU 转发 ADAS 信号给视觉处理芯片)

FW 还有一个特殊路径 mcu\_write\_to\_vip\_msg: ADAS 相关信号从 SOC 到 MCU 后, MCU 再转发给 VIP (视觉图像处理器)。SOC COM 发送到 "/x\_dom\_can\_rt/write\_to\_vip\_msgs" 主题, MCU 接收后通过 CAN 转发。

五、关键数据流汇总

